

# Model Refinement

## 1. Overview of Refinement Strategy

In Assignment 2, non-linear ensemble models such as Random Forest (Accuracy: 0.682, ROC-AUC: 0.73) and XGBoost (Accuracy: 0.627, ROC-AUC: 0.67) showed the highest performance. However, current metrics indicate that the models still need to improve their generalization capacity and reduce the margin of error in potability (class 1) predictions in order to achieve high accuracy.

Our refining strategy at this stage goes beyond simply increasing raw data; it focuses on optimizing model complexity, preventing overfitting, and maximizing information gain from critical features such as "Sulfate, pH, and Solids."

## 2. Model Evaluation (Current Baseline Analysis)

Before refining the models, we need to thoroughly examine the confusion matrices and error trends obtained from Assignment 2:

- **Post-Class Imbalance Situation:** The SMOTE implementation resolved the "0.0 Recall" error in Logistic Regression. However, even in the best model, Random Forest, 59 potable water samples were classified as "non-potable" (False Negative) and 69 non-potable samples were classified as "potable" (False Positive).
- **Cost of Misclassification:** In line with SDG 6 (Clean Water and Sanitation), predicting contaminated water as "potable" (False Positive) poses the greatest risk to human health. Therefore, refining the balance between Precision and Recall is essential.

## 3. Refinement Techniques

To further push the performance limits of the models, the following advanced techniques had to be applied at this stage:

### A. Advanced Data Balancing and Cleaning Integration

The precision cleaning performed in Assignment 2 may have resulted in data loss. To correct this, during the refining process:

- **SMOTE-Tomek Connections Integration:** This integration goes beyond simply generating synthetic data (SMOTE) and also incorporates the Tomek Connections technique to clarify boundaries between classes and remove noise.

This provides clearer decision boundaries for models that examine local data patterns, such as KNN and Decision Tree.

## B. Regularization

To prevent ensemble models from overfitting:

- **For XGBoost L1 (Alpha) and L2 (Lambda) Regularization:** This prevents the model from memorizing noisy interactions between chemical parameters..
- **Random Forest Pruning:** Decision limits are softened by limiting the maximum depth of the trees (max\_depth).

## 4. Hyperparameter Tuning (Advanced Search)

The basic GridSearchCV/RandomizedSearchCV operations performed in the previous stage have been refined in this stage to focus on more specific hyperparameters using coarse-to-fine searches.

| Model         | Assignment 2 Settings       | Assignment 3 Refined Parameters                                                                                               | Objective                                                                                         |
|---------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Random Forest | Default / Wide Search Range | n_estimators: [100, 200, 300]<br><br>max_depth: [8, 12, 15]<br><br>min_samples_split: [5, 10]                                 | To reduce tree complexity and lower the variance on the test set.                                 |
| XGBoost       | Basic Boosting              | learning_rate: [0.01, 0.05, 0.1]<br><br>max_depth: [4, 6, 8]<br><br>subsample: [0.7, 0.8]<br><br>colsample_bytree: [0.7, 0.8] | To slow down the gradient descent to prevent getting stuck in local minima and improve stability. |

## 5. Cross-Validation Strategy

To guarantee model stability, our standard 5-Fold Cross-Validation strategy has been refined as follows:

- **Stratified 5-Fold Cross-Validation:** The SMOTE process has been converted into a pipeline (Suhas-Pipeline) structure so that it is applied within each fold.
- **Data Leakage Prevention:** Data leakage of synthetic data into validation folds has been prevented. This prevents artificially high Accuracy and F1-Score metrics and verifies their robustness relative to real-world data.

## 6. Feature Selection

Our Feature Importance analysis in Figure 1 clearly shows that not all parameters are equally weighted in determining water quality:

**Most Important Drivers:** pH, Sulfate, and Solids have the highest importance scores.

**Least Important Drivers:** Trihalomethanes and Turbidity are the variables with the least impact on potability.

### Refinement Decision:

Recursive Feature Elimination (RFE) was applied to reduce the dimensionality coefficient of the model and eliminate noise. Trihalomethanes, which has the lowest variance and importance score, was eliminated from the model, ensuring that the models focus only on the main drivers (pH, Sulfate, Solids, Chloramines). This simplification optimized the accuracy of distance/line-based models, especially KNN and Logistic Regression.

## **PART B: Test Submission & Real-World Data Evaluation**

### **1. Overview**

The primary purpose of the test submission phase is to evaluate the generalization capacity and real-world applicability of our refined machine learning model. Following feedback to increase project originality, we transitioned from the standard Kaggle dataset to authentic, regional water quality data.

- **Scope and Purpose:** This phase assesses how well the optimized pipeline handles raw, unlabelled environmental sensor data, ensuring the model can accurately enforce safety regulations in a real-world scenario.
- **Pipeline Summary:** The pipeline involves extracting raw data from the US EPA Water Quality Portal, performing strategic feature selection to handle data sparsity, engineering a deterministic target variable based on World Health Organization (WHO) standards, and finally passing the data through our refined StandardScaler -> SMOTE -> RandomForest pipeline to generate test predictions.

### **2. Data Preparation for Testing**

Preparing real-world environmental data requires significantly different considerations compared to synthetic datasets.

**Preprocessing Steps:** Data was extracted for the New York region. Because raw sensor data arrives in a "Long Format," we pivoted it into a "Wide Format" to align with our model's expected input.

**Handling Sparsity & Differences from Training:** Unlike the Kaggle dataset where all 9 chemicals were present (or easily imputed), real-world monitoring stations rarely measure all parameters simultaneously. Requiring all 9 features would result in a completely empty dataset. To avoid introducing extreme bias through massive mean imputation, we relied on our Part A Feature Importance results and isolated the top two predictive drivers: pH and Sulfate.

**Target Variable Creation:** Real data lacks a predefined "Potability" label. We engineered this target deterministically using strict WHO thresholds (pH between 6.5 - 8.5, and Sulfate  $\leq$  250 mg/L). Samples meeting these criteria were labeled as 1 (Potable), and those failing were labeled as 0 (Not Potable), yielding 753 clean, usable records.

### 3. Model Application

The refined model architecture was applied to this newly engineered real-world dataset to test its inference capabilities.

- **Inference Pipeline:** The data was split using a rigorous 80/20 Train-Test split. We utilized the ImbPipeline framework from the imbalanced-learn library. This ensures that StandardScaler and SMOTE (used to balance the 467 Not Potable vs. 286 Potable samples) are applied strictly within the training folds, preventing any data leakage into the test set. The optimized Random Forest classifier was then used to predict the test fold.

```
# Splitting features and target variable
X = df.drop("Potability", axis=1)
y = df["Potability"]

# 80/20 Train-Test split with stratification to maintain class distribution
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Constructing the optimized pipeline (StandardScaler -> SMOTE -> RandomForest)
final_pipeline = ImbPipeline([
    ('scaler', StandardScaler()), # Scaling features
    ('smote', SMOTE(random_state=42)), # Balancing classes only on training data
    ('classifier', RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)) # Refined RF model
])

# Training the pipeline
final_pipeline.fit(X_train, y_train)

# Applying the model to the test dataset (Inference)
y_pred_final = final_pipeline.predict(X_test)
y_prob_final = final_pipeline.predict_proba(X_test)[:, 1]
```

### 4. Test Metrics

The model's performance on the unseen EPA test dataset was evaluated using standard classification metrics:

- **Accuracy:** 1.000 (100%)
- **Precision:** 1.000 (100%)
- **Recall:** 1.000 (100%)
- **F1-Score:** 1.000 (100%)
- **Comparison & Overfitting Discussion:** In traditional machine learning, perfect 1.0 scores often indicate severe overfitting or data leakage. However, in this specific real-world deployment test, these scores are mathematically justified.

Because the Potability target variable was created using explicit, deterministic rules based on pH and Sulfate, our non-linear Random Forest model successfully reverse-engineered and mapped these exact decision boundaries with zero error. This validates that the model has perfectly learned the environmental safety thresholds without generating any False Positives.

## 5. Model Deployment

To transition this model from a local environment to a real-world application for sustainable water management (SDG 6), the following deployment steps are structured:

- **Deployment Environment:** The trained pipeline is serialized (saved) using joblib or pickle and wrapped in a RESTful API framework such as **FastAPI** or **Flask**. The application can then be containerized using Docker and deployed to cloud platforms like AWS (EC2/Lambda), Heroku, or Render.
- **External System Integration:** Once deployed, the API can receive real-time JSON payloads containing pH and Sulfate readings directly from regional IoT water quality sensors or environmental agency databases. The model will instantly process the payload and return a "Potable" or "Not Potable" prediction, enabling automated, real-time public health alerts.

## 6. Code Implementation

```
# Deterministic Target Engineering based on WHO thresholds (pH 6.5-8.5 and Sulfate <= 250)
def label_potability(row):
    try:
        ph = float(row.get('pH', np.nan))
        sulfate = float(row.get('Sulfate', np.nan))
        if pd.isna(ph) or pd.isna(sulfate): return np.nan
        if (6.5 <= ph <= 8.5) and (sulfate <= 250): return 1 # Potable
        else: return 0 # Not Potable
    except:
        return np.nan

# Applying the rules to create the target variable for real-world data
df_pivot['Potability'] = df_pivot.apply(label_potability, axis=1)

# Function to calculate and print evaluation metrics
def evaluate_model(name, y_test, y_pred, y_prob):
    print(f"--- {name} Sonuçları ---")
    print(f"Accuracy : {accuracy_score(y_test, y_pred):.3f}")
    print(f"Precision: {precision_score(y_test, y_pred, zero_division=0):.3f}")
    print(f"Recall : {recall_score(y_test, y_pred, zero_division=0):.3f}")
    print(f"F1-score : {f1_score(y_test, y_pred, zero_division=0):.3f}")
    print(f"ROC-AUC : {roc_auc_score(y_test, y_prob):.3f}\n")

# Evaluating the final deployment-ready model on the test dataset
evaluate_model("Optimized Pipeline (SMOTE + Random Forest)", y_test, y_pred_final, y_prob_final)
```

## Conclusion

The model refinement and test submission phases successfully demonstrated the viability of transitioning from synthetic datasets to real-world, noisy environmental data. The primary challenge encountered was extreme data sparsity in regional sensor readings. This was strategically overcome by leveraging the Feature Importance insights from Part A, reducing the dimensionality to the most critical drivers: pH and Sulfate. By applying our refined, SMOTE-enhanced Random Forest pipeline to this new data, the model achieved perfect classification performance, flawlessly mapping the deterministic safety thresholds. The framework is now robust, highly accurate, and fully prepared for API deployment to support real-time water potability analysis.

## **PART C: Deployment (Overview, Model Serialization, Model Serving, API Integration, Security Considerations, Monitoring & Logging)**

### **1. Overview**

The deployment phase represents the transition of the developed machine learning model from an experimental notebook environment into a real-world usable decision-support system. In Part A, the model was refined through advanced preprocessing, feature selection, hyperparameter tuning, cross-validation, and model complexity control. In Part B, the refined model was tested on real-world water quality data obtained from the US EPA Water Quality Portal. The final model used two major predictive features, pH and Sulfate, because real-world environmental sensor data showed high sparsity and did not consistently include all of the original Kaggle dataset variables.

The main objective of deployment is to make the trained model accessible to external users, agencies, or automated systems through a stable and secure application interface. In this project, the deployed system is designed to receive water quality measurements, process them using the same preprocessing steps used during training, and return a prediction about whether the given water sample is Potable or Not Potable.

The deployment process is especially important because the project is related to SDG 6: Clean Water and Sanitation. A water potability prediction system should not remain only as a local machine learning experiment. Instead, it should be transformed into a practical tool that can support public health authorities, municipalities, environmental agencies, and water quality monitoring systems.

The final deployed model is based on an optimized machine learning pipeline consisting of:

Input features:

- pH
- Sulfate

Preprocessing:

- StandardScaler for numerical feature scaling
- SMOTE for balancing the minority class during training

Model:

- Optimized Random Forest Classifier

Output:

- Potable / Not Potable prediction
- Prediction probability
- Risk interpretation message

The model achieved perfect classification performance on the engineered EPA test dataset. However, this result should be interpreted carefully. Since the target variable was created using deterministic WHO-based thresholds for pH and Sulfate, the Random Forest model was able to learn these decision boundaries successfully. Therefore, the deployment phase should not only focus on technical implementation but also on safe usage, input validation, monitoring, and continuous model evaluation.

In a real-world environment, the deployment architecture can be summarized as follows:

1. Environmental sensor or user submits pH and Sulfate values.
2. API receives the data as a JSON request.
3. Input validation checks whether the values are realistic and complete.
4. The saved model pipeline processes the input.
5. The Random Forest model generates the prediction.
6. The API returns the prediction result and probability.
7. Logs are stored for future monitoring and performance tracking.
8. Model predictions are periodically reviewed against updated water quality standards and real laboratory results.

This structure allows the model to be used not only for batch prediction but also for real-time water safety monitoring.

## 2. Model Serialization

Model serialization is the process of saving the trained machine learning model to a file so that it can be loaded and reused later without retraining. This is a critical step in deployment because training a model every time a prediction is needed would be inefficient, time-consuming, and unnecessary.

In this project, the final model is not only a standalone Random Forest classifier. It is a complete machine learning pipeline that includes preprocessing steps and the classifier. Therefore, the entire pipeline should be serialized rather than saving only the classifier. This ensures that future predictions are processed in exactly the same way as the training data.

The deployment-ready pipeline includes:

- StandardScaler
- SMOTE used during training
- Optimized Random Forest Classifier

Although SMOTE is only applied during training and not during real-time inference, it is still part of the training pipeline structure. During deployment, the API loads the trained pipeline and applies the necessary transformation steps to the new input data before producing a prediction.

The most suitable serialization tools for this project are:

- joblib
- pickle

Between these two options, joblib is preferred for scikit-learn models because it is more efficient when storing models containing NumPy arrays and tree-based estimators such as Random Forest.

## 3. Model Serving

Model serving refers to making the trained model available for prediction requests. In this project, the best approach is to serve the model through a RESTful API. A REST API allows different systems to communicate with the model using standard HTTP requests.

For this project, two possible frameworks can be used:

- Flask
- FastAPI

Although Flask is simple and widely used, FastAPI is more suitable for this project because it provides automatic data validation, better performance, and automatic API documentation through Swagger UI. Since this project requires structured numerical input such as pH and Sulfate values, FastAPI's built-in validation system is highly useful.

The model serving architecture can be structured as follows:

User -> FastAPI Endpoint -> Input Validation -> Loaded Machine Learning Pipeline -> Prediction Result -> JSON Response

#### **4. API Integration**

API integration explains how the deployed model can communicate with external systems. In a real-world water quality monitoring scenario, the model may receive data from several possible sources.

Possible data sources include:

- IoT water quality sensors
- Municipal water monitoring systems
- Environmental agency databases
- Laboratory information systems
- Manual data entry dashboards
- Mobile or web applications

The API should be designed to receive real-time measurements in JSON format. For example, a regional IoT sensor may send pH and Sulfate values every hour. The deployed model can instantly evaluate the sample and return a potability prediction.

A possible integration flow is:

Water Quality Sensor -> Data Gateway -> Water Potability API -> Prediction Result -> Dashboard / Alert System / Database

The API can also be integrated into a dashboard. For example, a dashboard may show:

- Latest water quality readings
- Potability prediction by region
- Number of unsafe samples
- Average pH level by station
- Sulfate trends over time
- Warning alerts for risky locations

This would make the model more useful for decision-makers.

A possible dashboard workflow is:

API Prediction Logs -> Database -> Power BI / Streamlit / Tableau Dashboard -> Public Health Monitoring

## **5. Containerization and Cloud Deployment**

After building the API, the next step is deploying it to a server or cloud environment. For this purpose, the application should be containerized using Docker.

Docker allows the model, API code, dependencies, and configuration files to run consistently across different machines. This is important because a model that works on a local computer may fail on another system due to package version differences.

The recommended project structure is:

water-potability-api/

- app.py
- water\_potability\_model\_v1.pkl
- requirements.txt
- Dockerfile
- model\_metadata.json
- README.md

The requirements.txt file should include the required packages:

Fastapi, uvicorn, pandas, numpy, scikit-learn, imbalanced-learn, joblib, pydantic

For cloud deployment, several platforms can be used:

- Render
- Heroku
- AWS EC2
- AWS Lambda
- Google Cloud Run
- Azure App Service

For an academic project, **Render** or **Heroku** would be easier to use because they require less infrastructure management. For a real public health application, **AWS**, **Azure**, or **Google Cloud** would be more appropriate due to scalability, monitoring, and security features.

## 6. Security Considerations

Security is an essential part of deployment, especially because this project is related to water safety and public health. If the system is used by municipalities or environmental agencies, incorrect access, manipulation, or misuse of the API may lead to serious consequences.

The first security requirement is **input validation**. The API should reject unrealistic values. For example, pH must be between 0 and 14. Sulfate should not be negative. If invalid values are accepted, the model may produce misleading predictions.

Example validation rules:

- $0 \leq \text{pH} \leq 14$
- Sulfate  $\geq 0$
- Missing values are not allowed
- Non-numeric inputs are rejected

The second requirement is **authentication**. The API should not be open to everyone if it is used in a real-world environment. API keys or token-based authentication can be used.

Example:

- Authorization: Bearer <API\_TOKEN>

The third requirement is **HTTPS encryption**. All API communication should be encrypted using HTTPS. This prevents attackers from reading or modifying sensor data during transmission.

The fourth requirement is **rate limiting**. Without rate limiting, an attacker may send thousands of requests and overload the system. Rate limiting protects the API from abuse.

Example rule:

- Maximum 100 requests per minute per API key

The fifth requirement is **logging and auditability**. Every prediction request should be logged with timestamp, input values, prediction result, confidence score, and model version. This allows future investigation if an incorrect prediction occurs.

The sixth requirement is **model file protection**. The serialized model file should not be publicly downloadable. It should be stored securely in the server environment.

The seventh requirement is **error handling**. The API should not expose internal error messages to users. Instead, it should return clear and safe error responses.

Security is especially important in this project because false predictions may have public health implications. The model should support decision-making, but it should not fully replace laboratory testing or regulatory inspection.

## 7. Monitoring and Logging

Monitoring and logging are necessary to ensure that the deployed model continues to work correctly after deployment. A machine learning model may perform well during testing but degrade over time in production due to changes in data distribution. This is known as data drift or model drift.

In this project, monitoring should focus on both technical performance and model behavior.

Technical monitoring includes:

- API uptime
- Response time
- Number of requests
- Error rate
- Server CPU and memory usage
- Failed prediction requests

Model monitoring includes:

- Distribution of incoming pH values
- Distribution of incoming Sulfate values
- Number of Potable predictions
- Number of Not Potable predictions
- Average confidence score
- Frequency of extreme values

- Comparison with laboratory-confirmed results if available

For example, during testing, the EPA dataset contained 753 usable observations, and the pH values showed a strong relationship with potability. The Assignment\_3 output also showed that pH had a high correlation with the engineered Potability variable. Therefore, in deployment, the system should continuously check whether incoming pH distributions are similar to the training data distribution. If future data is significantly different, the model may need to be retrained.

Monitoring also helps identify unusual patterns. For example:

- If almost all predictions become “Not Potable,” there may be a real water quality issue or sensor malfunction.
- If Sulfate values suddenly become extremely high, the data pipeline should be checked.
- If API response time increases, the server may need scaling.
- If confidence scores decrease over time, the model may require retraining.

Alert rules can also be created:

- If Not Potable predictions exceed 30% in one region within 24 hours, send alert.
- If  $\text{pH} < 6.5$  or  $\text{pH} > 8.5$ , mark sample as risky.
- If Sulfate  $> 250$  mg/L, mark sample as risky.
- If API error rate  $> 5\%$ , notify technical team.

## 8. Model Maintenance and Retraining Strategy

Deployment is not the final stage of a machine learning project. After the model is deployed, it should be maintained and improved over time.

For this water potability project, retraining may be required under the following conditions:

- New EPA data becomes available.
- Additional chemical features become consistently available.
- Water quality standards are updated.
- The model receives too many uncertain predictions.
- Real laboratory results show disagreement with model predictions.
- Regional water characteristics change over time.

A recommended retraining cycle would be:

Every 3 or 6 months:

1. Collect new sensor and laboratory data.
2. Clean and preprocess the data.
3. Compare new data distribution with old training data.
4. Retrain the Random Forest model.
5. Evaluate the new model.
6. Compare it with the previous deployed version.
7. Deploy only if the new model performs better.

## **9. Limitations of the Deployment**

Although the deployment design is technically strong, the project has several limitations that should be clearly stated.

First, the final real-world EPA model uses only two features: pH and Sulfate. This decision was necessary because of data sparsity in real-world monitoring stations. However, water potability is actually affected by many additional chemical and biological factors, such as bacteria, heavy metals, organic pollutants, turbidity, chloramines, conductivity, and other contaminants.

Second, the target variable was not originally available in the EPA dataset. It was created deterministically using pH and Sulfate thresholds. Because of this, the model's perfect performance does not necessarily mean that it can detect all types of unsafe water. It means that the model successfully learned the engineered threshold rules.

Third, the model should not be used as the only authority for drinking water safety. In real public health practice, machine learning predictions should support laboratory testing, expert review, and official water quality regulations.

Fourth, the model may not generalize perfectly to regions outside New York unless it is retrained with data from those regions. Different regions may have different geological, industrial, and environmental conditions.

Fifth, sensor errors may affect predictions. If pH or Sulfate readings are incorrect, the model output will also be unreliable. Therefore, sensor calibration and data quality checks are necessary.

These limitations do not make the deployment invalid. Instead, they show that the model should be used responsibly as a decision-support system rather than a final regulatory authority.

## **10. Conclusion**

The deployment phase transforms the refined water potability model into a practical and accessible prediction system. The final model, based on pH and Sulfate values, can be serialized using joblib, served through a FastAPI-based REST API, containerized with Docker, and deployed to a cloud environment such as Render, Heroku, AWS, Azure, or Google Cloud.

The deployed API can receive real-time water quality measurements from environmental sensors, agency databases, or manual user input. It can then return a Potable or Not Potable prediction together with a confidence score and risk message. This makes the system suitable for supporting water quality monitoring and public health decision-making.

However, because the model's target variable was created using deterministic pH and Sulfate thresholds, the perfect test performance should be interpreted carefully. The model successfully learned the engineered safety boundaries, but it should still be monitored, validated, and updated with new real-world data. Security, input validation, monitoring, logging, model versioning, and retraining are essential for responsible deployment.

Overall, the proposed deployment structure provides a complete machine learning production pipeline for water potability prediction. It connects model refinement, real-world testing, API serving, security, and monitoring into a single practical system. This supports the broader objective of SDG 6 by contributing to faster, more automated, and data-driven water safety assessment.